

Árvores B+

Gabriel Gonçalves de Castro¹

¹ Departamento de Computação
Centro Federal de Educação Tecnológica de Minas Gerais (CEFET-MG)
Leopoldina – MG – Brasil

`connect.goncalves@gmail.com`

Resumo. *Este trabalho apresenta um estudo sobre a Árvore B+, uma estrutura de dados amplamente utilizada em sistemas de indexação para memória secundária. São discutidas suas propriedades estruturais, as diferenças em relação à Árvore B tradicional e a análise de complexidade assintótica das operações fundamentais de busca, inserção, remoção e consulta de intervalo. Por fim, são apresentadas aplicações práticas em sistemas de arquivos e bancos de dados, além de exemplos resolvidos ilustrando o comportamento da estrutura durante inserções e remoções.*

1. Introdução

Com o crescimento exponencial do volume de dados gerados por sistemas modernos, a indexação eficiente em dispositivos de armazenamento secundário tornou-se um requisito crítico para o desempenho de Bancos de Dados e Sistemas de Arquivos. Como o acesso ao disco ou SSD é significativamente mais lento do que o acesso à memória RAM, minimizar o número de operações de leitura e escrita (**Disk I/Os**) é o principal objetivo das estruturas de indexação.

Embora a Árvore B tradicional tenha solucionado parte desse problema ao introduzir múltiplos caminhos e nós paginados, sua variante, a Árvore B+, consolidou-se como o padrão de fato para a indústria de software. A principal limitação da Árvore B reside no fato de que os dados (ou ponteiros para os registros) são armazenados em qualquer nó da estrutura. Isso reduz o espaço disponível para as chaves de busca nos nós internos, diminuindo o fator de ramificação e aumentando a altura da árvore.

A Árvore B+ otimiza essa arquitetura ao separar completamente o índice dos dados. Os nós internos funcionam estritamente como um guia de roteamento, contendo apenas chaves de separação e ponteiros para outros nós. Todos os registros de dados são armazenados exclusivamente nos nós folhas. Além disso, as folhas são interconectadas sequencialmente por meio de ponteiros, formando uma lista ligada. Essa modificação permite duas grandes vantagens: maior densidade de chaves por nó interno (árvores mais baixas) e uma eficiência extrema em buscas por intervalo (**range queries**), dispensando a necessidade de percursos em ordem (**in-order traversal**) complexos por toda a árvore.

Este trabalho apresenta um estudo detalhado sobre a Árvore B+, abordando suas propriedades estruturais, análise de complexidade assintótica e presença em tecnologias atuais. Adicionalmente, serão apresentados cenários práticos e exemplos resolvidos detalhando o comportamento da estrutura durante operações de inserção e remoção.

2. História

A Árvore B foi introduzida em [Bayer and McCreight 1972], por pesquisadores dos Boeing Scientific Research Laboratories. O objetivo era criar um índice eficiente para arquivos de acesso aleatório de grande porte armazenados em disco.

Não existe um artigo principal que introduziu a Árvore B+. A noção de manter todos os dados nas folhas surgiu repetidamente como uma variante interessante da Árvore B original [Comer 1979]. Douglas Comer registra, em sua pesquisa sobre Árvores B, que a Árvore B+ já era utilizada no software de acesso a dados VSAM da IBM, com referência a um artigo publicado pela empresa em 1973 [Comer 1979].

Donald E. Knuth contribuiu de forma significativa ao apresentar uma análise matemática rigorosa das Árvores B em [Knuth 1998], formalizando suas propriedades e algoritmos. Knuth é frequentemente associado à estrutura por essa influência, mas não é seu criador.

3. A Estrutura de uma Árvore B+

Uma Árvore B+ T é composta por nós, sendo $T.raiz$ o nó raiz.

Todo nó x pertencente a T possui os seguintes atributos básicos:

- $x.n$: o número de chaves atualmente armazenadas no nó x .
- $x.chave_1 \leq x.chave_2 \leq \dots \leq x.chave_{x.n}$: as chaves armazenadas em ordem crescente.
- $x.folha$: um valor booleano que indica se x é um nó folha.

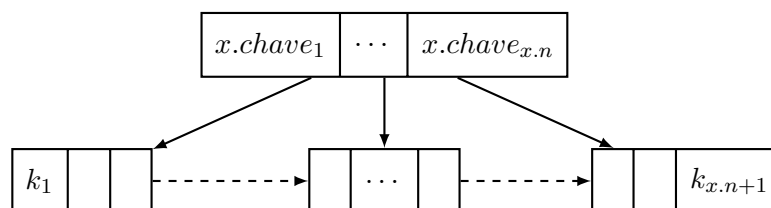
A estrutura complementar de x depende do seu tipo:

- Caso x seja um nó interno ($x.folha = \text{FALSO}$): Possui $x.n + 1$ ponteiros para seus filhos, denotados por $x.filho_1, x.filho_2, \dots, x.filho_{x.n+1}$.
- Caso x seja um nó folha ($x.folha = \text{VERDADEIRO}$): Possui um ponteiro $x.próximo$ que aponta para o próximo nó folha à direita (lista ligada), além dos ponteiros para os dados (ou valores) associados a cada chave.

Se considerarmos um nó interno x , com seus ponteiros $x.filho_1, x.filho_2, \dots, x.filho_{x.n+1}$, a seguinte condição é verdadeira:

$$k_1 \leq x.chave_1 \leq k_2 \leq x.chave_2 \leq \dots \leq x.chave_{x.n} \leq k_{x.n+1}$$

onde k_i são as chaves presentes nas subárvores que cada ponteiro $x.filho_i$ aponta.



4. Propriedades dos nós de uma Árvore B+

Uma Árvore B+ deve satisfazer as seguintes condições de balanceamento:

- Todo caminho do nó raiz até um nó folha qualquer deve possuir o mesmo tamanho. Com isso, todas as folhas possuem a mesma profundidade, que é a própria altura da árvore.
- Os nós têm limites superiores e inferiores para o número de chaves que podem conter. Expressamos esses limites em termos de um inteiro fixo $t \geq 2$, denominado **grau mínimo** da árvore.
 - Todo nó, exceto a raiz, deve ter no mínimo $t - 1$ chaves. Assim, todo nó interno, exceto a raiz, têm no mínimo t filhos.
 - Se a árvore é não-vazia, a raiz deve conter no mínimo uma chave.
 - Todo nó pode conter no máximo $2t - 1$ chaves. Portanto, um nó interno pode ter no máximo $2t$ filhos.

Por satisfazer as condições de balanceamento dadas, podemos provar que a altura de uma árvore B+:

$$h \leq \log_t(n)$$

onde n é o número de chaves armazenadas na árvore.

5. Operações Fundamentais

5.1. Busca

A busca em uma Árvore B+ consiste em um percurso descendente da raiz até as folhas. Nos nós internos, as chaves funcionam como separadores de roteamento: para cada nó visitado, encontramos o índice i tal que $chave < x.chave_i$, seguindo o ponteiro $x.filho_i$. Esse processo se repete até atingirmos um nó folha, onde os dados de fato residem.

Ao chegar na folha, realizamos uma busca linear pelas chaves armazenadas. A chave procurada existe na árvore se e somente se encontrarmos $x.chave_i = chave$ para algum índice válido i .

O algoritmo BUSCAR descreve esse procedimento.

5.2. Análise da Busca

A operação de busca percorre exatamente um caminho da raiz até uma folha, visitando $O(\log_t n)$ nós. Em cada nó, realiza-se uma busca linear entre as chaves, custando $O(t)$ comparações. Portanto, a complexidade de tempo da busca é $O(t \log_t n)$. Como t é uma constante definida pelo grau mínimo da árvore, essa expressão é equivalente a $O(\log n)$.

Como o algoritmo é iterativo, a complexidade de espaço adicional é $O(1)$. Vale ressaltar que em implementações recursivas, o espaço ocupado pela pilha de chamadas contribui para a complexidade de espaço, elevando-a para $O(\log_t n)$.

5.3. Inserção

A inserção em uma Árvore B+ inicia com uma busca pela folha destino, seguindo um percurso descendente. Ao localizar a folha correta, a chave é inserida em sua posição ordenada. Os algoritmos INSERIR e INSERIRNÃOOCHEIO descrevem esse procedimento.

Algorithm 1 Busca em uma Árvore B+

```
1: procedure BUSCAR(chave)
2:   atual  $\leftarrow T.raiz$ 
3:   while  $\neg atual.folha$  do
4:      $i \leftarrow 1$ 
5:     while  $i \leq atual.n \wedge chave \geq atual.chave[i]$  do
6:        $i \leftarrow i + 1$ 
7:     end while
8:
9:     atual  $\leftarrow atual.filho[i]$ 
10:  end while
11:
12:   $i \leftarrow 1$ 
13:  while  $i \leq atual.n \wedge chave > atual.chave[i]$  do
14:     $i \leftarrow i + 1$ 
15:  end while
16:  return  $i \leq atual.n \wedge chave = atual.chave[i]$ 
17: end procedure
```

Algorithm 2 Inserção em uma Árvore B+

```
1: procedure INSERIR(chave)
2:   if  $T.raiz.n = 2t - 1$  then
3:     DIVIDIRRAIZ
4:   end if
5:   INSERIRNÃOCHEIO(T.raiz, chave)
6: end procedure
```

Algorithm 3 Inserção em nó não cheio

```
1: procedure INSERIRNÃOCHIEIO( $x, chave$ )
2:   if  $x.folha$  then
3:      $i \leftarrow x.n$ 
4:     while  $i \geq 1 \wedge chave < x.chave[i]$  do
5:        $x.chave[i + 1] \leftarrow x.chave[i]$ 
6:        $i \leftarrow i - 1$ 
7:     end while
8:      $x.chave[i + 1] \leftarrow chave$ 
9:      $x.n \leftarrow x.n + 1$ 
10:  else
11:     $i \leftarrow x.n$ 
12:    while  $i \geq 1 \wedge chave < x.chave[i]$  do
13:       $i \leftarrow i - 1$ 
14:    end while
15:     $i \leftarrow i + 1$ 
16:
17:    if  $x.filho[i].n = 2t - 1$  then
18:      if  $x.filho[i].folha$  then
19:        DIVIDIRFOLHA( $x, i$ )
20:      else
21:        DIVIDIRINTERNO( $x, i$ )
22:      end if
23:      if  $chave > x.chave[i]$  then
24:         $i \leftarrow i + 1$ 
25:      end if
26:    end if
27:
28:    INSERIRNÃOCHIEIO( $x.filho[i], chave$ )
29:  end if
30: end procedure
```

Alguns possíveis problemas podem acontecer durante a inserção, por isso tomaremos algumas medidas preventivas. Enquanto estivermos descendo pela árvore, qualquer nó cheio encontrado é dividido antes de prosseguir, garantindo que o pai sempre tenha espaço para receber uma nova chave.

A divisão pode ocorrer de três formas diferentes: divisão de folha, divisão de nó interno e divisão de raiz. Na **divisão de folha**, as chaves são redistribuídas entre dois nós e a primeira chave do novo nó direito é **copiada** ao pai como separador, permanecendo também na folha. Na **divisão de nó interno**, a chave mediana é **promovida** ao pai e removida do nó dividido. Na **divisão de raiz**, um novo nó raiz é criado, aumentando a altura da árvore em um nível.

Algorithm 4 Divisão de folha

```

1: procedure DIVIDIRFOLHA(pai, idx)
2:   esq  $\leftarrow$  pai.filho[idx]
3:   dir  $\leftarrow$  novo Nó
4:   dir.folha  $\leftarrow$  VERDADEIRO
5:   dir.n  $\leftarrow$  t - 1
6:
7:   for i  $\leftarrow$  1 até t - 1 do
8:     dir.chave[i]  $\leftarrow$  esq.chave[t + i]
9:   end for
10:
11:  esq.n  $\leftarrow$  t
12:  dir.prox  $\leftarrow$  esq.prox
13:  esq.prox  $\leftarrow$  dir
14:
15:  for i  $\leftarrow$  pai.n descendo até idx do
16:    pai.filho[i + 1]  $\leftarrow$  pai.filho[i]
17:  end for
18:  pai.filho[idx + 1]  $\leftarrow$  dir
19:
20:  for i  $\leftarrow$  pai.n descendo até idx do
21:    pai.chave[i + 1]  $\leftarrow$  pai.chave[i]
22:  end for
23:  pai.chave[idx]  $\leftarrow$  dir.chave[1]
24:
25:  pai.n  $\leftarrow$  pai.n + 1
26: end procedure

```

5.4. Análise da Inserção

A operação de inserção percorre exatamente um caminho da raiz até uma folha, visitando $O(\log_t n)$ nós. Em cada nó, realiza-se uma busca linear e, possivelmente, uma divisão, ambas custando $O(t)$ operações. Portanto, a complexidade de tempo da inserção é $O(t \log_t n)$, equivalente a $O(\log n)$ para t constante.

Como o algoritmo é iterativo na descida e as divisões são realizadas localmente, a complexidade de espaço auxiliar é $O(1)$.

Algorithm 5 Divisão de nó interno

```
1: procedure DIVIDIRINTERNO(pai, idx)
2:   esq  $\leftarrow$  pai.filho[idx]
3:   dir  $\leftarrow$  novo Nó
4:   dir.folha  $\leftarrow$  FALSO
5:   dir.n  $\leftarrow$  t - 1
6:
7:   for i  $\leftarrow$  1 até t - 1 do
8:     dir.chave[i]  $\leftarrow$  esq.chave[t + i]
9:   end for
10:
11:  for i  $\leftarrow$  1 até t do
12:    dir.filho[i]  $\leftarrow$  esq.filho[t + i]
13:  end for
14:
15:  esq.n  $\leftarrow$  t - 1
16:
17:  for i  $\leftarrow$  pai.n descendo até idx do
18:    pai.filho[i + 1]  $\leftarrow$  pai.filho[i]
19:  end for
20:  pai.filho[idx + 1]  $\leftarrow$  dir
21:
22:  for i  $\leftarrow$  pai.n descendo até idx do
23:    pai.chave[i + 1]  $\leftarrow$  pai.chave[i]
24:  end for
25:  pai.chave[idx]  $\leftarrow$  esq.chave[t]
26:
27:  pai.n  $\leftarrow$  pai.n + 1
28: end procedure
```

Algorithm 6 Divisão da raiz

```
1: procedure DIVIDIRRAIZ
2:   r  $\leftarrow$  T.raiz
3:   T.raiz  $\leftarrow$  novo Nó
4:   T.raiz.folha  $\leftarrow$  FALSO
5:   T.raiz.n  $\leftarrow$  0
6:   T.raiz.filho[1]  $\leftarrow$  r
7:
8:   if r.folha then
9:     DIVIDIRFOLHA(T.raiz, 1)
10:  else
11:    DIVIDIRINTERNO(T.raiz, 1)
12:  end if
13: end procedure
```

5.5. Remoção

A remoção em uma Árvore B+ também segue um percurso descendente até a folha onde a chave reside. Ao localizá-la, a chave é removida e as posições subsequentes são deslocadas.

Assim como na inserção, problemas estruturais podem ocorrer durante a descida. Para evitá-los, adotamos uma estratégia preventiva: antes de entrar em um filho com apenas $t - 1$ chaves, garantimos que ele possua chaves suficientes por meio de GARANTIRMÍNIMO. Esse procedimento tenta primeiro redistribuir chaves de um irmão adjacente por meio de uma rotação. Caso nenhum irmão tenha chaves suficientes, funde o filho com um de seus irmãos.

Algorithm 7 Remoção em uma Árvore B+

```
1: procedure REMOVE(chave)
2:   REMOVENÓ(T.raiz, chave)
3:
4:   if T.raiz.n = 0  $\wedge$   $\neg$ T.raiz.folha then
5:     T.raiz  $\leftarrow$  T.raiz.filho[1]
6:   end if
7: end procedure
```

Quando um filho possui apenas $t - 1$ chaves, o procedimento GARANTIRMÍNIMO tenta redistribuir chaves de um irmão com chaves suficientes por meio de uma rotação. A **rotação à direita** empresta a última chave do irmão esquerdo, enquanto a **rotação à esquerda** empresta a primeira chave do irmão direito. Caso nenhuma rotação seja possível, o filho é fundido com um irmão adjacente.

Na rotação, o comportamento difere entre folhas e nós internos. Em folhas, a chave emprestada é movida diretamente e o separador no pai é atualizado. Em nós internos, o separador do pai desce para o filho e a chave do irmão sobe para o pai, junto com a transferência do ponteiro correspondente.

Na fusão, o comportamento também difere entre folhas e nós internos. Em folhas, as chaves do nó direito são copiadas para o esquerdo e a lista ligada é atualizada. Em nós internos, o separador do pai desce para o nó esquerdo junto com as chaves e ponteiros do direito. Em ambos os casos, o separador é removido do pai.

5.6. Análise da Remoção

A remoção também percorre um único caminho da raiz até uma folha, visitando $O(\log_t n)$ nós. Em cada nível, o procedimento GARANTIRMÍNIMO realiza no máximo uma rotação ou fusão, ambas com custo $O(t)$. Portanto, a complexidade de tempo da remoção é $O(t \log_t n)$, equivalente a $O(\log n)$ para t constante.

Assim como na inserção, a complexidade de espaço auxiliar é $O(1)$ para a implementação iterativa, e $O(\log_t n)$ para implementações recursivas, devido à pilha de chamadas.

Algorithm 8 Remoção recursiva

```
1: procedure REMOVERNÓ( $x, chave$ )
2:   if  $x.folha$  then
3:      $i \leftarrow 1$ 
4:     while  $i \leq x.n \wedge chave > x.chave[i]$  do
5:        $i \leftarrow i + 1$ 
6:     end while
7:
8:     if  $i > x.n \vee chave \neq x.chave[i]$  then return
9:     end if
10:
11:    for  $j \leftarrow i$  até  $x.n - 1$  do
12:       $x.chave[j] \leftarrow x.chave[j + 1]$ 
13:    end for
14:     $x.n \leftarrow x.n - 1$ 
15:  else
16:     $i \leftarrow 1$ 
17:    while  $i \leq x.n \wedge chave \geq x.chave[i]$  do
18:       $i \leftarrow i + 1$ 
19:    end while
20:
21:    if  $x.filho[i].n \leq t - 1$  then
22:      GARANTIRMÍNIMO( $x, i$ )
23:
24:       $i \leftarrow 1$ 
25:      while  $i \leq x.n \wedge chave \geq x.chave[i]$  do
26:         $i \leftarrow i + 1$ 
27:      end while
28:    end if
29:
30:    REMOVERNÓ( $x.filho[i], chave$ )
31:  end if
32: end procedure
```

Algorithm 9 Garantir chaves mínimas

```
1: procedure GARANTIRMÍNIMO( $x, i$ )
2:   if  $i > 1 \wedge x.filho[i - 1].n \geq t$  then
3:     ROTACIONARDIREITA( $x, i$ )
4:   else if  $i \leq x.n \wedge x.filho[i + 1].n \geq t$  then
5:     ROTACIONARÉQUERDA( $x, i$ )
6:   else if  $i \leq x.n$  then
7:     FUNDIRFILHOS( $x, i$ )
8:   else
9:     FUNDIRFILHOS( $x, i - 1$ )
10:  end if
11: end procedure
```

Algorithm 10 Rotação à direita

```
1: procedure ROTACIONARDIREITA( $x, i$ )
2:    $filho \leftarrow x.filho[i]$ 
3:    $irmão \leftarrow x.filho[i - 1]$ 
4:
5:   for  $j \leftarrow filho.n$  descendo até 1 do
6:      $filho.chave[j + 1] \leftarrow filho.chave[j]$ 
7:   end for
8:
9:   if  $filho.folha$  then
10:     $filho.chave[1] \leftarrow irmão.chave[irmão.n]$ 
11:     $x.chave[i - 1] \leftarrow filho.chave[1]$ 
12:  else
13:    for  $j \leftarrow filho.n + 1$  descendo até 1 do
14:       $filho.filho[j + 1] \leftarrow filho.filho[j]$ 
15:    end for
16:     $filho.chave[1] \leftarrow x.chave[i - 1]$ 
17:     $filho.filho[1] \leftarrow irmão.filho[irmão.n + 1]$ 
18:     $x.chave[i - 1] \leftarrow irmão.chave[irmão.n]$ 
19:  end if
20:
21:   $filho.n \leftarrow filho.n + 1$ 
22:   $irmão.n \leftarrow irmão.n - 1$ 
23: end procedure
```

Algorithm 11 Rotação à esquerda

```
1: procedure ROTACIONARESGUERDA( $x, i$ )
2:    $filho \leftarrow x.filho[i]$ 
3:    $irmão \leftarrow x.filho[i + 1]$ 
4:
5:   if  $filho.folha$  then
6:      $filho.chave[filho.n + 1] \leftarrow irmão.chave[1]$ 
7:     for  $j \leftarrow 1$  até  $irmão.n - 1$  do
8:        $irmão.chave[j] \leftarrow irmão.chave[j + 1]$ 
9:     end for
10:     $x.chave[i] \leftarrow irmão.chave[1]$ 
11:   else
12:      $filho.chave[filho.n + 1] \leftarrow x.chave[i]$ 
13:      $filho.filho[filho.n + 2] \leftarrow irmão.filho[1]$ 
14:      $x.chave[i] \leftarrow irmão.chave[1]$ 
15:     for  $j \leftarrow 1$  até  $irmão.n - 1$  do
16:        $irmão.chave[j] \leftarrow irmão.chave[j + 1]$ 
17:     end for
18:     for  $j \leftarrow 1$  até  $irmão.n$  do
19:        $irmão.filho[j] \leftarrow irmão.filho[j + 1]$ 
20:     end for
21:   end if
22:
23:    $filho.n \leftarrow filho.n + 1$ 
24:    $irmão.n \leftarrow irmão.n - 1$ 
25: end procedure
```

Algorithm 12 Fusão de filhos

```
1: procedure FUNDIRFILHOS( $x, idx$ )
2:    $esq \leftarrow x.filho[idx]$ 
3:    $dir \leftarrow x.filho[idx + 1]$ 
4:
5:   if  $esq.folha$  then
6:     for  $i \leftarrow 1$  até  $dir.n$  do
7:        $esq.chave[esq.n + i] \leftarrow dir.chave[i]$ 
8:     end for
9:      $esq.n \leftarrow esq.n + dir.n$ 
10:     $esq.prox \leftarrow dir.prox$ 
11:  else
12:     $esq.chave[esq.n + 1] \leftarrow x.chave[idx]$ 
13:    for  $i \leftarrow 1$  até  $dir.n$  do
14:       $esq.chave[esq.n + 1 + i] \leftarrow dir.chave[i]$ 
15:    end for
16:    for  $i \leftarrow 1$  até  $dir.n + 1$  do
17:       $esq.filho[esq.n + 1 + i] \leftarrow dir.filho[i]$ 
18:    end for
19:     $esq.n \leftarrow esq.n + dir.n + 1$ 
20:  end if
21:
22:  for  $i \leftarrow idx$  até  $x.n - 1$  do
23:     $x.chave[i] \leftarrow x.chave[i + 1]$ 
24:     $x.filho[i + 1] \leftarrow x.filho[i + 2]$ 
25:  end for
26:   $x.n \leftarrow x.n - 1$ 
27: end procedure
```

5.7. Consulta de Intervalo (Range Query)

Por conta de sua organização interna, as árvores B+ possuem uma grande vantagem nas consultas de intervalo (*Range Query*). Como todos os dados estão nas folhas, e elas possuem um comportamento bem parecido com uma lista ligada, conseguimos buscar os elementos de um determinado intervalo [mínimo, máximo] com grande facilidade, sem depender de percursos em ordem das árvores tradicionais.

Para começarmos uma busca em um intervalo, precisamos primeiro chegar na folha que guarda o valor mínimo que estamos buscando. Ao chegar lá, basta percorrer essa lista ligada até atingir o valor máximo ou até a lista encadeada das folhas chegar ao fim. O pseudocódigo a seguir demonstra esse procedimento.

Algorithm 13 Consulta de Intervalo

```
1: procedure INTERVALO(mínimo, máximo)
2:   atual  $\leftarrow$  T.raiz
3:
4:   while  $\neg$ atual.folha do
5:     i  $\leftarrow$  1
6:     while  $i \leq \textit{atual.n} \wedge \textit{mínimo} \geq \textit{atual.chave}[i]$  do
7:       i  $\leftarrow$  i + 1
8:     end while
9:
10:    atual  $\leftarrow$  atual.filho[i]
11:  end while
12:
13:  resultado  $\leftarrow$  nova LISTA
14:  while atual  $\neq$  NULL do
15:    for i  $\leftarrow$  1 até atual.n do
16:      if atual.chave[i] > máximo then
17:        return resultado
18:      end if
19:      if atual.chave[i]  $\geq$  mínimo then
20:        resultado.inserir(atual.chave[i])
21:      end if
22:    end for
23:
24:    atual  $\leftarrow$  atual.próximo
25:  end while
26:  return resultado
27: end procedure
```

5.8. Análise da Consulta de Intervalo

A consulta por intervalo percorre um único caminho da raiz até uma folha, visitando $O(\log_t n)$ nós. Ao chegar nas folhas, percorremos a lista encadeada coletando os elementos do intervalo [*mínimo*, *máximo*]. Seja *m* o número de chaves que satisfazem essa

condição. Como cada nó folha armazena no máximo $2t - 1$ chaves, o número de folhas visitadas é $O(m/t)$, e o custo total nessa etapa é $O(m)$.

Portanto, a complexidade de tempo da consulta por intervalo é $O(m + \log_t n)$.

A complexidade de espaço auxiliar para percorrer os nós é $O(1)$, pois a implementação é iterativa. O espaço adicional para o resultado depende de como ele é tratado: se passado como referência, o custo é $O(1)$; se instanciado internamente, o custo é $O(m)$.

6. Aplicações Práticas e Sistemas

A Árvore B+ consolidou-se como a estrutura de indexação padrão em sistemas de software modernos, presente tanto em sistemas de arquivos quanto em bancos de dados relacionais e NoSQL.

6.1. Sistemas de Arquivos

Diversos sistemas de arquivos amplamente utilizados adotam Árvores B+ para indexação de metadados. ReiserFS, NSS, XFS, JFS, ReFS e BFS utilizam essa estrutura para indexação de metadados; o BFS também a emprega para armazenamento de diretórios [Giampaolo 1999]. O NTFS utiliza Árvores B+ para indexação de diretórios e metadados relacionados à segurança. O EXT4 utiliza *extent trees*, uma estrutura derivada da Árvore B+, para indexação de extensões de arquivos. O APFS utiliza Árvores B+ para armazenar mapeamentos entre identificadores de objetos do sistema de arquivos e suas localizações em disco, bem como para registros do sistema de arquivos, incluindo diretórios [Apple Inc. 2020].

6.2. Bancos de Dados

Sistemas gerenciadores de bancos de dados relacionais como IBM Db2, Informix, Microsoft SQL Server, Oracle, Sybase ASE e SQLite utilizam Árvores B+ para índices de tabelas, cada um com variações e extensões sobre a estrutura básica [Ramakrishnan and Gehrke 2003]. Sistemas NoSQL como CouchDB e Tokyo Cabinet também adotam Árvores B+ para acesso e armazenamento de dados.

Em bancos de dados de alta dimensionalidade, a Árvore B+ é utilizada na construção do método de busca indexada *iDistance*, que realiza buscas pelos k vizinhos mais próximos (kNN) em espaços métricos de alta dimensão [Jagadish et al. 2005]. Os dados são particionados e cada partição recebe um valor de índice, permitindo que as consultas sejam mapeadas para buscas em intervalos unidimensionais, acelerando significativamente a varredura sequencial.

6.3. Memória Não Volátil

A Árvore B+ também tem sido adotada em sistemas de memória de acesso aleatório não volátil (NVRAM) como técnica principal de acesso à memória em sistemas de Internet das Coisas (IoT), devido ao seu baixo consumo estático de energia e alta solidez de memória de célula. Com estratégias avançadas sobre frequências de folhas e pontos de referência muito utilizados, a Árvore B+ demonstra resultados significativos no aumento da durabilidade de sistemas de banco de dados [Dharamjeet et al. 2021].

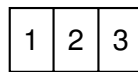
7. Exemplos Resolvidos

Ambos exemplos serão resolvidos com $t = 2$. Ou seja, a árvore contém no mínimo 1 chave e no máximo 3 chaves.

7.1. Exemplo 1: Inserção com Split

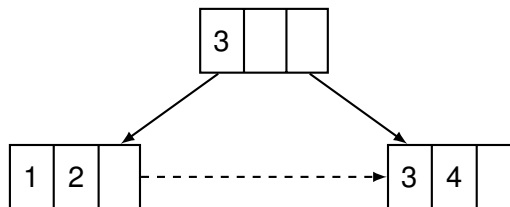
Inserimos as chaves 1, 2, 3, 4, 5, 6 em uma árvore B+ vazia. A estratégia preventiva divide qualquer nó com 3 chaves antes de descer nele.

Após inserir 1, 2, 3:

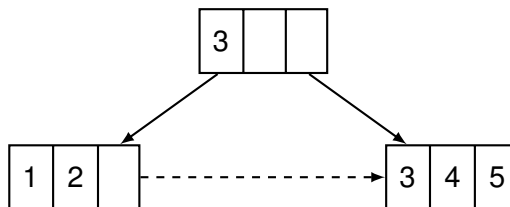


Inserir 4 (raiz cheia): split preventivo antes de descer.

A raiz é dividida: a chave 3 é **copiada** ao novo pai (comportamento de folha).

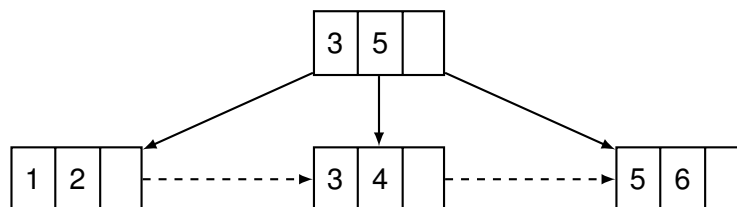


Após inserir 5:



Inserir 6 (folha direita cheia): split preventivo.

A folha $[3, 4, 5]$ é dividida. A chave 5 é **copiada** ao pai.

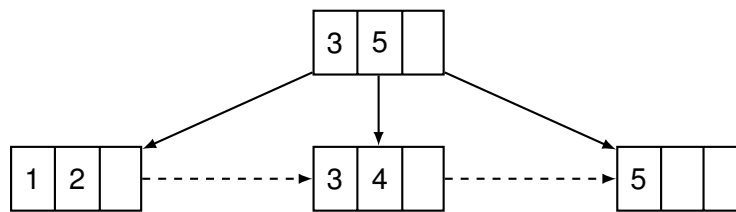


Esta é a árvore final após as 6 inserções.

7.2. Exemplo 2: Remoção com Redistribuição

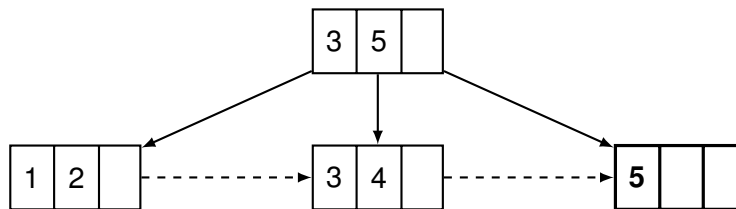
Partindo da árvore do Exemplo 1, removemos as chaves 6 e depois 5.

Remover 6 é simples: a folha $[5, 6]$ fica com $[5]$, ainda no mínimo ($t - 1 = 1$).

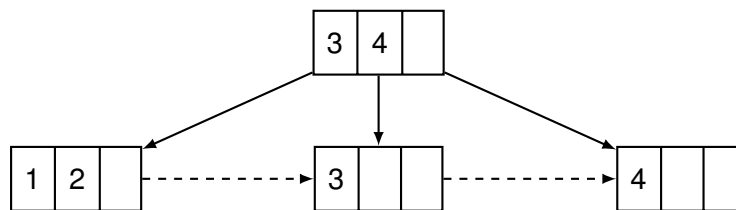


Agora removemos 5: a folha [5] ficaria vazia ($0 < t - 1 = 1$). O irmão esquerdo [3, 4] tem 2 chaves, ou seja, quantidade suficiente para emprestar. Executamos uma **rotação à direita**: 4 migra para a folha deficiente, e o separador no pai é atualizado de 5 para 4.

Antes:



Após remover 5 (rotação à direita):

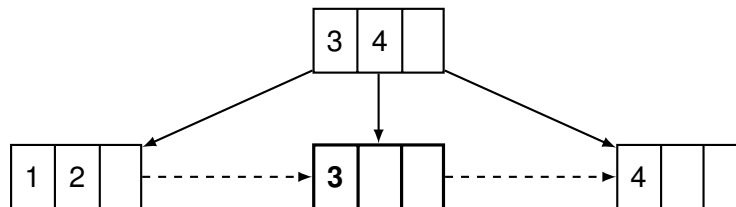


7.3. Exemplo 3: Remoção com Fusão

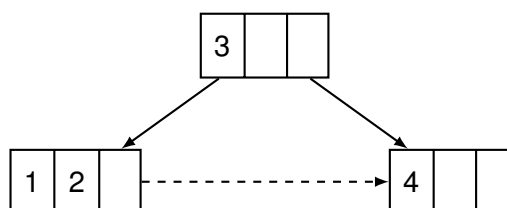
Partindo da árvore do Exemplo 2, removemos a chave 3.

A folha [3] ficaria vazia após a remoção ($0 < t - 1 = 1$). O irmão imediato direito [4] tem 1 chave, ou seja, não pode emprestar. Executamos **fusão**: as folhas [3] e [4] se unem em [4], e o separador 4 é removido do pai.

Antes:



Após remover 3 (fusão):



O pai perde o separador 4 e fica com 1 chave. Ainda válido pois $1 \geq t - 1 = 1$.

8. Conclusão

Este trabalho apresentou a Árvore B+, discutindo sua estrutura, propriedades, operações fundamentais e aplicações práticas. A separação entre nós internos e folhas, aliada à lista ligada entre as folhas, torna a Árvore B+ superior à Árvore B tradicional em cenários que envolvem buscas por intervalo e acesso sequencial a dados.

Sua adoção massiva em sistemas de arquivos e bancos de dados ao longo de décadas evidencia que a estrutura continua relevante e eficiente para os desafios modernos de armazenamento e recuperação de dados em memória secundária.

Referências

- Apple Inc. (2020). *Apple File System Reference*. Disponível em: <https://developer.apple.com/support/downloads/Apple-File-System-Reference.pdf>.
- Bayer, R. and McCreight, E. M. (1972). Organization and maintenance of large ordered indices. *Acta Informatica*, 1(3):173–189.
- Comer, D. (1979). The ubiquitous B-tree. *ACM Computing Surveys*, 11(2):121–137.
- Dharamjeet, Chen, T.-Y., Chang, Y.-H., Wu, C.-F., Lee, C.-H., and Shih, W.-K. (2021). Beyond write-reduction consideration: A wear-leveling-enabled B+-tree indexing scheme over an NVRAM-based architecture. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 40(12):2455–2466.
- Giampaolo, D. (1999). *Practical File System Design with the Be File System*. Morgan Kaufmann.
- Jagadish, H. V., Ooi, B. C., Tan, K.-L., Yu, C., and Zhang, R. (2005). iDistance: An adaptive B+-tree based indexing method for nearest neighbor search. *ACM Transactions on Database Systems*, 30(2):364–397.
- Knuth, D. E. (1998). *The Art of Computer Programming, Vol. 3: Sorting and Searching*. Addison-Wesley, 2 edition.
- Ramakrishnan, R. and Gehrke, J. (2003). *Database Management Systems*. McGraw-Hill, 3 edition.